Project 7 — Reusable component library

Gate: after Phase 5 (Design patterns). **Architectural focus:** headless/compound components, accessibility, API design, versioned distribution. **Estimated effort:** 35–45 h. **Stack:** React 19, TypeScript 5, Vite (library mode), Vitest, React Testing Library, `@testing-library/user-event`, `jest-axe` or `axe-core`, Storybook (or Ladle) for docs, Playwright (component or E2E), Changesets for versioning, `tsup` or Vite library mode for builds.

> Tooling versions (Storybook, Changesets, bundlers) move quickly. Use current docs for configuration; the design guidance below does not depend on them. Verified 2026-10-22 on React 19.3, TypeScript 5.9, Vitest 3 and Vite 8.3: the core utilities, `Slot`, `Button`, `Tabs`, the polymorphic `Box` type and `useControllableState` typecheck under `strict`; the library-mode config below builds and its output paths match the `exports` map; behavior claims in the notes were run in jsdom.

1. Project brief

Build `@acme/ui`: a small, production-grade component library of eight components used by several teams, plus a docs site. It consists of two tiers:

1. **Headless primitives** (behavior, state, ARIA, keyboard; no styling).
2. **Styled components** built on the primitives (tokens + CSS variables).

Consumers must be able to use the styled tier out of the box, drop to the headless tier when the design diverges, and never fork the library.

Components

Component	Pattern emphasis	Key a11y reference
`Button` / `IconButton`	Polymorphic (`as` / `asChild`), ref as prop, loading state	Native `button` semantics, `aria-disabled` vs `disabled`
`Dialog`	Compound, portal, focus trap, scroll lock, controlled/uncontrolled	APG Dialog (Modal) pattern
`Tabs`	Compound, roving tabindex, controlled/uncontrolled, orientation	APG Tabs pattern
`Select` (listbox) or `Combobox`	Headless hook + compound, typeahead, virtual focus	APG Listbox / Combobox patterns
`Tooltip`	Provider + timing, focus and hover, `aria-describedby`	WCAG 1.4.13 content on hover or focus
`Menu` (dropdown)	Compound, roving focus, typeahead, submenus optional	APG Menu Button pattern
`TextField`	Slots (label, hint, error), form association	Label/description wiring via ids

Component	Pattern emphasis	Key a11y reference
`Toast`	Imperative API + provider, live regions, timers pause on hover/focus	`role="status"` / `alert`, WCAG 2.2.1 timing

Functional requirements

ID	Requirement
F1	Every stateful component supports **controlled and uncontrolled** usage with the same prop names (`value` / `defaultValue` / `onValueChange`; `open` / `defaultOpen` / `onOpenChange`).
F2	Compound APIs (`Tabs.Root`, `Tabs.List`, `Tabs.Trigger`, `Tabs.Panel`) with context-based wiring and a **clear error** when a part is used outside its Root.
F3	Polymorphism: `Button` renders as `a`, `Link` from a router, or any element via `asChild` / `as`, with correct prop typing.
F4	`ref` is a regular prop on every component (React 19: no `forwardRef`), typed to the underlying element.
F5	All components merge consumer `className`, `style`, event handlers (consumer handler runs first, may `preventDefault` to opt out), and pass through unknown DOM props.
F6	Theming via CSS custom properties (design tokens). No runtime CSS-in-JS. Dark mode and `prefers-reduced-motion` supported.
F7	Ships ESM (+ types) with `sideEffects` correctly declared; each component is importable via a subpath and tree-shakeable.
F8	SSR-safe: no `window` / `document` access during render; stable `useId` ids; hydration warnings absent.
F9	A docs site (Storybook or similar) with usage, API tables, accessibility notes, and live examples including the controlled variants.
F10	Semantic versioning with Changesets; a written **deprecation policy** and one demonstrated deprecation.

Non-functional requirements

ID	Requirement
N1	WCAG 2.2 AA behaviors: keyboard operability, visible focus, name/role/value, target size, no keyboard traps outside intended modal focus trap.
N2	Each component contributes < 3 KB gzip (excluding shared core) except `Select` / `Menu` / `Dialog` (< 6 KB). Verify with `size-limit`.
N3	Zero runtime dependencies other than `react` / `react-dom` as peers. (Positioning libraries are an extension; decide and record.)
N4	Public API surface is explicit (`exports` map). Internals are not importable.
N5	100% of public props documented and typed; no `any` in exported types.

---

2. Learning objectives

1. Design component APIs that survive growth: composition over configuration (Composition over inheritance, Patterns for reusable libraries).

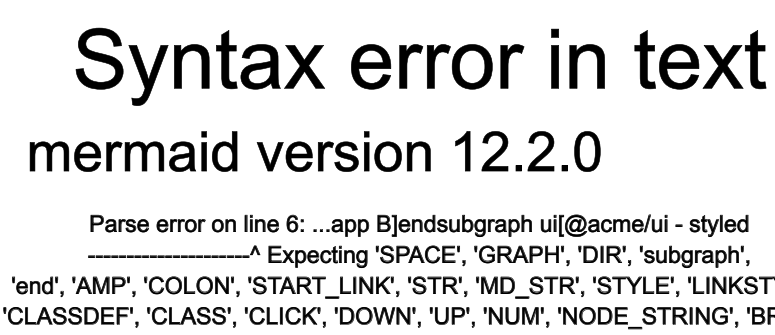
2. Implement compound components with context (Compound components, Provider pattern).
3. Separate behavior from presentation (Headless components, Behavior vs presentation).
4. Build correct controlled/uncontrolled hybrids (Controlled component pattern, Controlled and uncontrolled inputs).
5. Write polymorphic and slot-based APIs with sound types (Polymorphic components, Slot patterns).
6. Encode complex interaction in a state machine or reducer (State machine pattern, Reducer pattern).
7. Design error handling and misuse diagnostics (Error handling patterns).
8. Treat accessibility as architecture, not a checklist (Accessibility as architecture, Accessibility testing).
9. Test a component against its *contract* (Component contract testing, Visual regression).
10. Version, document, and govern a shared library (Design systems, Component libraries, Design system governance, Backward compatibility).
11. Recognize when a pattern is unnecessary (Patterns made less necessary, Custom hook patterns).

Other useful lessons: Higher-order components and Render props (know why you are *not* using them), Custom hooks, Context performance, `useId` in modern hooks, Monorepos and shared packages.

---

3. Recommended architecture

3.1 Three layers



- **Core (internal):** tiny utilities every component needs. Not part of the public API.
- **Primitives (public, unstyled):** behavior + ARIA + keyboard. They render real elements with `data-*` state attributes (`data-state="open"` , `data-disabled`) as styling hooks.
- **Styled (public):** thin wrappers that apply classes/tokens and fix defaults. If the styled `Tabs` needs more than ~40 lines beyond the primitive, behavior has leaked into the wrong layer.

3.2 Principles for the API

1. **Prefer composition to props.** `<Dialog.Title>` beats `title="..."` : consumers can wrap, reorder, or replace parts without a new prop for each case.
2. **State is controllable at the root only.** One owner per piece of state.
3. **Props mean what DOM props mean.** `disabled` , `onClick` , `className` behave as on native elements; do not invent synonyms.
4. **Data attributes, not class toggles,** expose state to CSS.

5. **Accessibility is not optional configuration.** If a `Dialog` lacks a title, warn in development (`aria-labelledby` required).
6. **Fail loudly in dev, quietly in production.** Misuse errors/warnings are stripped or tree-shaken in production builds.
7. **Public surface is a contract.** Everything exported is semver-governed; everything else is private.

3.3 Decision: why not HOCs, render props, or class inheritance

Hooks and composition made HOCs and most render props unnecessary for library behavior: they obscure the element tree, collide on prop names, and type poorly. Render props remain justified when a consumer must render *per item* with library-provided state (e.g., a virtualized `Select` item renderer). Record these judgments in your ADR (Patterns made less necessary).

---

4. Folder structure

```
acme-ui/
├─ package.json                      # exports map, sideEffects, peerDependencies
├─ tsconfig.json, tsconfig.build.json
├─ vite.config.ts                    # library mode, multiple entries
├─ .changeset/
├─ src/
│  ├─ core/                          # internal
│  │  ├─ compose.ts                  # composeEventHandlers, composeRefs
│  │  ├─ context.ts                  # createStrictContext
│  │  ├─ slot.tsx                    # asChild support
│  │  ├─ dev.ts                      # warn/invariant stripped in prod
│  │  └─ types.ts                    # PolymorphicProps, ComponentPropsWithRef utilities
│  ├─ hooks/
│  │  ├─ useControllableState.ts
│  │  ├─ useRovingFocus.ts
│  │  ├─ useTypeahead.ts
│  │  ├─ useFocusTrap.ts
│  │  ├─ useScrollLock.ts
│  │  ├─ useDismiss.ts               # Escape + outside press
│  │  └─ usePresence.ts              # mount/unmount for exit animations
│  ├─ primitives/
│  │  ├─ dialog/ dialog.tsx, index.ts, dialog.test.tsx
│  │  ├─ tabs/
│  │  ├─ select/
│  │  ├─ menu/
│  │  ├─ tooltip/
│  │  └─ toast/
│  ├─ components/                    # styled tier
│  │  ├─ button/ button.tsx, button.css, button.stories.tsx, button.test.tsx
│  │  ├─ dialog/
│  │  ├─ tabs/
│  │  └─ ...
│  ├─ styles/
│  │  ├─ tokens.css                  # --acme-color-*, --acme-space-*, --acme-radius-*
│  │  └─ reset.css (optional, opt-in)
│  └─ index.ts                       # re-exports styled tier only
├─ docs/                             # Storybook config, MDX pages
├─ tests/
│  ├─ a11y/ keyboard-contracts/
│  └─ ssr/                           # renderToString smoke tests
├─ e2e/                              # Playwright against Storybook
└─ DEPRECATION.md, CONTRIBUTING.md, ADR/
```

`package.json` essentials:

```
{
  "name": "@acme/ui",
  "type": "module",
  "sideEffects": ["**/*.css"],
  "exports": {
    ".": { "types": "./dist/index.d.ts", "import": "./dist/index.js" },
    "./primitives/*": { "types": "./dist/primitives/*.d.ts", "import":
"./dist/primitives/*.js" },
    "./tokens.css": "./dist/tokens.css",
    "./package.json": "./package.json"
  },
  "peerDependencies": { "react": "^19.0.0", "react-dom": "^19.0.0" },
  "files": ["dist"]
}
```

With the build config in §6.10, Vite 8 emits `dist/index.js` and `dist/primitives/tabs.js` (plus a shared chunk), which are the files the `exports` map points at; `types` must stay the first condition. Still verify the `exports` conditions and types resolution against your bundler and TypeScript `moduleResolution` setting (`bundler` or `node16` / `nodenext`). A wrong `exports` map is the most common reason a library works in its own repo but not in consumers.

---

5. Milestones

#	Milestone	Deliverable	Exit criterion
M0	Design doc	API sketches for all 8 components as *usage code only* (before implementation), plus principles from 3.2	Peer review: can you write the 3 most common consumer scenarios per component?
M1	Core utilities	`composeEventHandlers`, `composeRefs`, `createStrictContext`, `useControllableState`, `Slot`	100% branch coverage; unit tests for edge cases
M2	Button + polymorphism	Button, IconButton, `asChild`, loading, type tests	`tsd` / `expect-type` tests prove prop inference
M3	Tabs	Compound, roving focus, controlled/uncontrolled, orientation	Keyboard contract test table passes
M4	Dialog	Portal, focus trap/restore, scroll lock, Escape/outside press, nested dialogs	a11y + keyboard + focus-restore tests
M5	TextField + Tooltip	Slots, id wiring; hover/focus tooltip with delay and Escape dismissal	axe clean; WCAG 1.4.13 test
M6	Menu + Select	Typeahead, roving or virtual focus	Contract tests for Arrow/Home/End/typeahead/Escape
M7	Toast	Provider + `toast()` API, queue, live region, pause on hover/focus	Timing tests with fake timers

#	Milestone	Deliverable	Exit criterion
M8	Styling tier	Tokens, dark mode, reduced motion, high-contrast sanity check	Visual regression baselines
M9	Packaging	Build, exports map, types, size-limit, SSR smoke test, install test in a throwaway consumer app	`npm pack` consumed successfully by a fresh Vite app
M10	Docs and release	Storybook, API tables, a11y notes, Changesets release, deprecation demo	Docs build in CI; one `@deprecated` prop shipped with a codemod or migration note

---

6. Recommended implementation

6.1 Core utilities

```
// core/compose.ts
import type { Ref, RefCallback } from 'react'

/** Run the consumer's handler first; skip ours if they called preventDefault. */
export function composeEventHandlers<E extends { defaultPrevented: boolean }>(
  theirs: ((e: E) => void) | undefined,
  ours: ((e: E) => void) | undefined,
) {
  return (e: E) => {
    theirs?.(e)
    if (!e.defaultPrevented) ours?.(e)
  }
}

/** Merge refs. React 19: callback refs may return a cleanup function. */
export function composeRefs<T>(...refs: Array<Ref<T> | undefined>): RefCallback<T> {
  return (node) => {
    const cleanups: Array<() => void> = []
    for (const ref of refs) {
      if (typeof ref === 'function') {
        const cleanup = ref(node)
        if (typeof cleanup === 'function') cleanups.push(cleanup)
        else cleanups.push(() => ref(null))
      } else if (ref) {
        ;(ref as { current: T | null }).current = node
        cleanups.push(() => { (ref as { current: T | null }).current = null })
      }
    }
    return () => cleanups.forEach((c) => c())
  }
}
```

Notes: In React 19, a callback ref may return a cleanup function; when it does, React calls the cleanup instead of calling the ref with `null`. The merged ref above always returns a cleanup that restores the legacy `null` semantics for each constituent ref. Verified on React 19.3: mixing a legacy callback ref, an object ref and a cleanup-style ref through `composeRefs` works, including under StrictMode, where the sequence is attach, detach (legacy ref receives `null`, cleanup ref runs its cleanup), attach. Also note that a *new* `composeRefs` result each render causes React to detach/attach on every render (verified: 3 attaches for a mount plus two re-renders); memoize with `useMemo` / `useCallback` where it matters (useRef and mutable references).

```
// core/context.ts
import { createContext, use } from 'react'

export function createStrictContext<T>(name: string, parentName: string) {
  const Ctx = createContext<T | null>(null)
  Ctx.displayName = name
  function useStrict(): T {
    const value = use(Ctx) // `use` can read context conditionally in React 19; useContext is
equivalent here
    if (value === null) throw new Error(`<${name}> must be used within <${parentName}>`)
    return value
  }
  return [Ctx, useStrict] as const
}
```

React 19 allows rendering `<Ctx value={...}>` directly (no `.Provider`); `.Provider` still works (Provider pattern, `usecontext-and-propagation`).

```tsx
// hooks/useControllableState.ts
import { useCallback, useRef, useState } from 'react'

interface Options<T> {
  value: T | undefined
  defaultValue: T
  onChange?: (next: T) => void
}

export function useControllableState<T>({ value, defaultValue, onChange }: Options<T>) {
  const [internal, setInternal] = useState(defaultValue)
  const isControlled = value !== undefined
  const current = isControlled ? value : internal

  // Detect controlled/uncontrolled flips in dev (a consumer bug, like React's own inputs).
  const wasControlled = useRef(isControlled)
  if (import.meta.env?.DEV && wasControlled.current !== isControlled) {
    console.warn('A component changed from ' + (wasControlled.current ? 'controlled to
uncontrolled' : 'uncontrolled to controlled') + '. Decide on one for its lifetime.')
  }

  const onChangeRef = useRef(onChange)
  onChangeRef.current = onChange // writing a ref during render: see note

  const setValue = useCallback(
    (next: T | ((prev: T) => T)) => {
      const resolved = typeof next === 'function' ? (next as (p: T) => T)(current) : next
      if (!isControlled) setInternal(resolved)
      if (!Object.is(resolved, current)) onChangeRef.current?.(resolved)
    },
    [isControlled, current],
  )
  return [current, setValue] as const
}
```

Review points in this hook (use them as discussion in your retrospective):

- Writing `onChangeRef.current` during render violates render purity under concurrent rendering (a render may be discarded); the conventional fix is assigning in an effect (or `useEffectEvent`, which is stable since React 19.2 and exported by 19.3; it is meant for effects, so it cannot be passed down or called during render, and it does not fit a callback you hand to children) (Component purity, Modern hooks and APIs). The trade-off: effect assignment delays visibility by one commit, which is invisible for event handlers called after commit.
- `import.meta.env?.DEV` is a bundler convention. In a published library, use a build-time replaced constant (`process.env.NODE_ENV` is the historical convention) so consumer bundlers can strip it. Decide, test in a consumer.

- Calling `onChange` *inside* a state updater function is a bug (updaters must be pure and are double-invoked in StrictMode). The code above computes `resolved` outside any updater to avoid that.
- A functional `next` is resolved against `current` at call time; batching two calls in one event handler will not chain. For primitives that only emit values, accept values not updaters; simpler and safer.

6.2 Slot / `asChild` and polymorphic typing

Two approaches to polymorphism:

Approach	Usage	Pros	Cons
`as` prop	`<Button as="a" href="/x">`	Simple mental model	Generic typing is heavy; `as={RouterLink}` props must be inferred
`asChild` (Slot)	`<Button asChild><a href="/x">…</a></Button>`	Child keeps its own typing and props; works with any component	Child must forward props and ref; runtime cloning

```tsx
// core/slot.tsx
import { Children, cloneElement, isValidElement, type HTMLAttributes, type ReactElement, type Ref } from 'react'
import { composeEventHandlers, composeRefs } from './compose'

interface SlotProps extends HTMLAttributes<HTMLElement> {
  children?: React.ReactNode
  ref?: Ref<HTMLElement>
}

export function Slot({ children, ref, ...slotProps }: SlotProps) {
  const child = Children.only(children)
  if (!isValidElement(child)) return null
  const childProps = child.props as Record<string, unknown> & { ref?: Ref<HTMLElement> }
  return cloneElement(child as ReactElement<Record<string, unknown>>, {
    ...mergeProps(slotProps, childProps),
    ref: composeRefs(ref, childProps.ref), // React 19: element ref lives in props.ref
  })
}

function mergeProps(slot: Record<string, any>, child: Record<string, any>) {
  const merged = { ...slot, ...child }
  for (const key of Object.keys(slot)) {
    if (/^on[A-Z]/.test(key) && typeof slot[key] === 'function' && typeof child[key] === 'function') {
      merged[key] = composeEventHandlers(child[key], slot[key]) // child (consumer) first
    }
  }
  if (slot.className || child.className) merged.className = [slot.className, child.className].filter(Boolean).join(' ')
  if (slot.style || child.style) merged.style = { ...slot.style, ...child.style }
  return merged
}
```

In React 19, `element.props.ref` is how a child element's ref is read; in React 18, it was `element.ref`. Pin your `peerDependencies` to the version you tested. `Children.only` and `cloneElement` are legacy-flavored APIs documented as having alternatives; they remain the common implementation for slots, but check the current docs for caveats.

The `as` variant, typed:

```tsx
// core/types.ts
import type { ComponentPropsWithRef, ElementType } from 'react'

export type PolymorphicProps<E extends ElementType, Own = {}> =
  Own & { as?: E } & Omit<ComponentPropsWithRef<E>, keyof Own | 'as'>
```

```tsx
export function Box<E extends ElementType = 'div'>({ as, ...rest }: PolymorphicProps<E, {
tone?: 'neutral' | 'danger' }>) {
  const Comp: ElementType = as ?? 'div'
  return <Comp {...rest} />
}
// <Box as="a" href="/x" /> compiles; <Box as="div" href="/x" /> does not.
```

Because React 19 makes `ref` an ordinary prop, `ComponentPropsWithRef` includes the right `ref` type and no `forwardRef` generic gymnastics are needed (Polymorphic components).

6.3 Button

```tsx
// components/button/button.tsx
import type { ComponentPropsWithRef } from 'react'
import { Slot } from '../../core/slot'

export interface ButtonProps extends ComponentPropsWithRef<'button'> {
  variant?: 'solid' | 'outline' | 'ghost'
  size?: 'sm' | 'md' | 'lg'
  loading?: boolean
  asChild?: boolean
}

export function Button({ variant = 'solid', size = 'md', loading, asChild, disabled,
className, children, onClick, ...rest }: ButtonProps) {
  const Comp = asChild ? Slot : 'button'
  const inert = disabled || loading
  return (
    <Comp
      {...rest}
      className={['acme-button', className].filter(Boolean).join(' ')}
      data-variant={variant}
      data-size={size}
      data-loading={loading || undefined}
      // `disabled` removes the button from the tab order. While loading we keep focus and
      // announce state via aria-disabled so keyboard users do not lose their place.
      disabled={asChild ? undefined : disabled}
      aria-disabled={inert || undefined}
      aria-busy={loading || undefined}
      onClick={(e: React.MouseEvent<HTMLButtonElement>) => (inert ? e.preventDefault() :
onClick?.(e))}
    >
      {children}
    </Comp>
  )
}
```

Caveat (verified): with `asChild`, the child's *own* `onClick` runs before the Button's guard (consumer handlers are composed first), so a `loading`/`disabled` Button does not stop a handler the child declares itself; only handlers passed to `Button` are blocked. Document that, or run the guard first. Design notes: `type` defaults to `"submit"` for native buttons inside forms; decide whether your Button defaults to `type="button"` (safer, differs from native) and document it. A loading button with `aria-disabled` and no spinner label hides the state from screen readers; pair with visually hidden text or `aria-live`.

6.4 Tabs (compound + roving focus)

```tsx
// primitives/tabs/tabs.tsx
import { useId, useMemo, useRef, type ComponentPropsWithRef, type KeyboardEvent } from 'react'
import { createStrictContext } from '../../core/context'
import { composeEventHandlers } from '../../core/compose'
import { useControllableState } from '../../hooks/useControllableState'

interface TabsCtx {
  baseId: string
  value: string
  setValue: (v: string) => void
  orientation: 'horizontal' | 'vertical'
}
const [TabsContext, useTabs] = createStrictContext<TabsCtx>('Tabs', 'Tabs.Root')

interface RootProps extends Omit<ComponentPropsWithRef<'div'>, 'defaultValue' | 'onChange'> {
  value?: string
  defaultValue?: string
  onValueChange?: (v: string) => void
  orientation?: 'horizontal' | 'vertical'
}

function Root({ value, defaultValue = '', onValueChange, orientation = 'horizontal', ...rest
}: RootProps) {
  const baseId = useId()
  const [current, setCurrent] = useControllableState({ value, defaultValue, onChange:
onValueChange })
  const ctx = useMemo(() => ({ baseId, value: current, setValue: setCurrent, orientation }),
[baseId, current, setCurrent, orientation])
  return (
    <TabsContext value={ctx}>
      <div {...rest} data-orientation={orientation} />
    </TabsContext>
  )
}

function List(props: ComponentPropsWithRef<'div'>) {
  const { orientation } = useTabs()
  return <div role="tablist" aria-orientation={orientation} {...props} />
}

function Trigger({ value, disabled, onKeyDown, onClick, ...rest }:
ComponentPropsWithRef<'button'> & { value: string }) {
  const ctx = useTabs()
  const selected = ctx.value === value
  const handleKeyDown = (e: KeyboardEvent<HTMLButtonElement>) => {
    const horizontal = ctx.orientation === 'horizontal'
```

```
    const next = { [horizontal ? 'ArrowRight' : 'ArrowDown']: 1, [horizontal ? 'ArrowLeft' :
'ArrowUp']: -1 }[e.key]
    const list = e.currentTarget.closest('[role="tablist"]')!
    const tabs = Array.from(list.querySelectorAll<HTMLButtonElement>
('[role="tab"]:not([disabled])'))
    const i = tabs.indexOf(e.currentTarget)
    let target: HTMLButtonElement | undefined
    if (next) target = tabs[(i + next + tabs.length) % tabs.length]
    else if (e.key === 'Home') target = tabs[0]
    else if (e.key === 'End') target = tabs.at(-1)
    if (target) { e.preventDefault(); target.focus(); target.click() } // automatic activation
  }
  return (
    <button
      {...rest}
      type="button"
      role="tab"
      id={`${ctx.baseId}-tab-${value}`}
      aria-selected={selected}
      aria-controls={`${ctx.baseId}-panel-${value}`}
      tabIndex={selected ? 0 : -1}
      data-state={selected ? 'active' : 'inactive'}
      disabled={disabled}
      onClick={composeEventHandlers(onClick, () => ctx.setValue(value))}
      onKeyDown={composeEventHandlers(onKeyDown, handleKeyDown)}
    />
  )
}

function Panel({ value, forceMount, ...rest }: ComponentPropsWithRef<'div'> & { value: string;
forceMount?: boolean }) {
  const ctx = useTabs()
  const active = ctx.value === value
  if (!active && !forceMount) return null
  return (
    <div
      {...rest}
      role="tabpanel"
      id={`${ctx.baseId}-panel-${value}`}
      aria-labelledby={`${ctx.baseId}-tab-${value}`}
      hidden={!active}
      tabIndex={0}
    />
  )
}
```

```
export const Tabs = { Root, List, Trigger, Panel }
```

Explain and critique:

- **Roving tabindex:** only the selected tab has `tabIndex=0`; arrows move focus within the list. The ids and `aria-controls` / `aria-labelledby` wiring come from `useId`, which is stable across SSR and hydration.
- **DOM queries for sibling lookup** (`querySelectorAll`) keep the sample short but couple behavior to DOM order and attribute selectors. The production-grade alternative is a registration collection (each Trigger registers a ref with the root) so order and disabled state are explicit and survive portals/wrappers. Implement this in your solution and compare (Behavior vs presentation).
- **Automatic vs manual activation:** APG allows both. Automatic (focus selects) is simplest but triggers panel changes on every arrow; manual activation (Space/Enter selects) is better when panels are expensive. Expose an `activationMode` prop.
- **Context re-render cost:** every Trigger and Panel consumes the whole context; any value change re-renders all of them. For a handful of tabs that is fine; for large lists split into state/dispatch contexts or use an external store (Context performance).
- **Missing:** RTL keyboard handling (swap Left/Right in `dir="rtl"`), `Delete` key behaviors if closable.

6.5 Dialog (portal, focus management, dismissal)

Requirements for a modal dialog (APG):

1. On open, focus moves into the dialog (first focusable, or a designated `autoFocus` element, or the container with `tabIndex={-1}`).
2. Tab/Shift+Tab are trapped inside.
3. Escape closes (unless disabled for destructive confirm flows, with justification).
4. On close, focus returns to the element that opened it (the trigger), or a sensible fallback if it unmounted.
5. Content behind is inert: `aria-hidden` / `inert` on siblings and scroll locked.
6. `role="dialog"` (or `alertdialog`), `aria-modal="true"`, labelled by the title, described by the description.

```tsx
// primitives/dialog/dialog.tsx (abridged)
function Root({ open, defaultOpen = false, onOpenChange, children }: RootProps) {
  const [isOpen, setOpen] = useControllableState({ value: open, defaultValue: defaultOpen,
onChange: onOpenChange })
  const triggerRef = useRef<HTMLElement | null>(null)
  const ids = { title: useId(), description: useId() }
  const ctx = useMemo(() => ({ isOpen, setOpen, triggerRef, ids }), [isOpen, setOpen,
ids.title, ids.description])
  return <DialogContext value={ctx}>{children}</DialogContext>
}

function Content({ children, ...rest }: ComponentPropsWithRef<'div'>) {
  const { isOpen, setOpen, triggerRef, ids } = useDialog()
  const ref = useRef<HTMLDivElement>(null)

  useFocusTrap(ref, isOpen)                       // cycles Tab within ref
  useScrollLock(isOpen)                           // counter-based so nested dialogs do not
unlock early
  useDismiss(ref, { onDismiss: () => setOpen(false), escape: true, outsidePress: true })

  useEffect(() => {
    if (!isOpen) return
    const previouslyFocused = document.activeElement as HTMLElement | null
    ref.current?.focus()
    return () => (triggerRef.current ?? previouslyFocused)?.focus()   // restore on
close/unmount
  }, [isOpen, triggerRef])

  if (!isOpen) return null
  return createPortal(
    <div data-dialog-overlay>
      <div
        {...rest}
        ref={ref}
        role="dialog"
        aria-modal="true"
        aria-labelledby={ids.title}
        aria-describedby={ids.description}
        tabIndex={-1}
        data-state="open"
      >
        {children}
      </div>
    </div>,
    document.body,
```

```
  )
}
```

Considerations the reference solution must address:

- **SSR:** `document.body` exists only on the client; the content renders nothing when closed, but a `defaultOpen` dialog on the server needs a deferred portal (mount after hydration, e.g., via a `mounted` state set in an effect) to avoid a mismatch (Hydration internals).
- **Exit animations** need `usePresence` to delay unmount until `animationend`; respect `prefers-reduced-motion`.
- **Inert siblings:** the `inert` attribute is now widely supported across current browsers (check your own support matrix) and `@types/react` 19 types it as a boolean attribute (`inert?: boolean`).
- **Native `<dialog>` alternative:** `showModal()` provides focus trapping, top-layer rendering, and Escape handling for free. It is a legitimate base for a modern library; trade-offs include styling the backdrop, animation, and testing in jsdom (limited support). Evaluate it in an ADR rather than defaulting to a hand-rolled trap (HTML spec on dialog).
- **Nested dialogs:** the trap and scroll lock need a stack so the topmost handles Escape/Tab.
- **Missing title:** in dev, warn if no `Dialog.Title` registers.

6.6 Tooltip, Menu, Select: shared hooks

- `useRovingFocus(containerRef, { orientation, loop })` is used by Tabs, Menu, and (for non-combobox listboxes) Select.
- `useTypeahead(items, onMatch)`: accumulate characters for ~500 ms, match by prefix of item text, ignoring disabled items.
- **Select:** choose between *real focus on options* (listbox with roving tabindex) and *virtual focus* (`aria-activedescendant` on a combobox input). The latter is required when focus must remain in a text input. Document the choice with the relevant APG pattern (component-library trade-offs).
- **Tooltip:** must be dismissible (Escape), hoverable (pointer can move onto the tooltip without it disappearing), and persistent until dismissed, hover leaves, or focus leaves (WCAG 1.4.13). It should only hold *supplementary* text; essential info belongs in visible text. Use `aria-describedby` on the trigger.
- **Positioning** (flipping, collision) is genuinely hard. Decision: dependency (e.g., Floating UI) vs a minimal in-house implementation limited to four placements. Record it against N3.

6.7 Toast

```
interface ToastOptions { id?: string; title: string; description?: string; duration?: number;
tone?: 'info' | 'success' | 'error' }

class ToastStore { /* queue + subscribe, usable outside React: toast.success(...) */ }
export const toast = { show: store.add, dismiss: store.remove }

export function Toaster() {
  const toasts = useSyncExternalStore(store.subscribe, store.getSnapshot,
store.getServerSnapshot)
  return (
    <section aria-label="Notifications" tabIndex={-1}>
      <ol role="list">
        {toasts.map((t) => <ToastItem key={t.id} toast={t} />)}
      </ol>
    </section>
  )
}
```

Requirements: use a live region that exists in the DOM *before* messages are inserted (screen readers announce changes to existing live regions more reliably than new ones); `role="alert"` only for urgent errors; timers pause on hover and focus; no auto-dismiss for toasts that contain actions unless the action is available elsewhere (WCAG 2.2.1 Timing Adjustable). The store is external so non-React code can emit toasts (useSyncExternalStore).

6.8 Dev diagnostics

```
// core/dev.ts
export function warn(condition: unknown, message: string) {
  if (process.env.NODE_ENV !== 'production' && !condition) console.warn(`[acme-ui]
${message}`)
}
```

Use for: missing `Dialog.Title`, controlled/uncontrolled flip, `Tabs.Trigger` value with no matching `Panel`, `Tooltip` trigger that is not focusable. These checks replace documentation lookups for common mistakes.

6.9 Styling contract

```
/* styles/tokens.css */
:root {
  --acme-color-bg: #fff;
  --acme-color-fg: #14161a;
  --acme-color-accent: #2557d6;
  --acme-focus-ring: 2px solid var(--acme-color-accent);
  --acme-radius-md: 6px;
  --acme-space-2: 8px;
}
@media (prefers-color-scheme: dark) { :root { --acme-color-bg: #14161a; --acme-color-fg:
#f2f4f8; } }
:root[data-theme='dark'] { --acme-color-bg: #14161a; --acme-color-fg: #f2f4f8; }

.acme-button { border-radius: var(--acme-radius-md); padding: var(--acme-space-2) 14px; }
.acme-button[data-variant='solid'] { background: var(--acme-color-accent); color: #fff; }
.acme-button:focus-visible { outline: var(--acme-focus-ring); outline-offset: 2px; }
.acme-button[aria-disabled='true'] { opacity: .6; cursor: not-allowed; }
@media (prefers-reduced-motion: reduce) { * { transition-duration: 0.01ms !important;
animation-duration: 0.01ms !important; } }
```

Verify contrast ratios of every token pair (4.5:1 text, 3:1 UI components) in both themes. The reduced-motion rule above is intentionally blunt; scope it to your own classes in a library so you do not alter consumers' CSS.

6.10 Build configuration

```
// vite.config.ts (library mode)
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import dts from 'vite-plugin-dts'
import { resolve } from 'node:path'

export default defineConfig({
  plugins: [react(), dts({ include: ['src'] })],
  build: {
    lib: {
      entry: { index: resolve(import.meta.dirname, 'src/index.ts'), 'primitives/tabs':
resolve(import.meta.dirname, 'src/primitives/tabs/index.ts') /* ... */ },
      formats: ['es'],
    },
    rollupOptions: { external: ['react', 'react-dom', 'react/jsx-runtime'] },
    cssCodeSplit: true,
  },
})
```

Plugin choices (`vite-plugin-dts` 5.x, `tsup` , `tsc` emit) and the "preserve modules" approach to maximize tree-shaking vary; verify with current docs, then prove tree-shaking by importing one component into a consumer and inspecting its bundle (Bundle analysis). Externalize `react/jsx-runtime` , otherwise you ship a second copy of the JSX runtime.

---

7. Design decisions to document

Decision	Options	Notes
Polymorphism	`as` vs `asChild`	`asChild` types better with arbitrary components but needs child cooperation. Pick one primary and justify.
Compound vs prop-config	`Tabs.Root/...` vs `<Tabs items={[...]}/>`	Compound for flexibility; offer a convenience prop-driven wrapper only if three consumers ask.
Focus strategy per widget	Roving tabindex vs `aria-activedescendant`	Dictated by whether DOM focus must stay elsewhere.
Collection registration	DOM queries vs React registration	DOM queries are small but brittle with portals/reorders.
Dialog implementation	Hand-rolled vs native `<dialog>`	See 6.5.
Positioning	Dependency vs in-house	Cost of collision handling vs N3.
Styling	CSS vars + data attributes vs CSS Modules vs runtime CSS-in-JS	Runtime CSS-in-JS conflicts with React 19 streaming/concurrent rendering concerns and adds bytes; CSS variables allow theming without re-rendering. This is a trade-off, not a universal ban.
Disabled semantics	`disabled` vs `aria-disabled`	`aria-disabled` keeps focusability; must block activation manually.
Context granularity	Single vs split	Split when consumers re-render too often (measure).
Versioning policy	What counts as breaking	Changing DOM structure, `data-*` attributes, default `type` , and focus behavior *are* breaking for consumers who style or test against them. Document them as part of the public contract (Backward compatibility).
Deprecation	Warn in dev, keep for one major, provide codemod/migration note	

---

8. Testing expectations

Strategy: test the **contract** each component promises, not its implementation (Component contract testing, Avoiding brittle tests).

Layer	What
Unit	`composeEventHandlers` (order, `preventDefault`), `composeRefs` (function ref, object ref, cleanup), `useControllableState` (controlled, uncontrolled, flip warning, `onChange` not called when value unchanged), `mergeProps`
Keyboard contract tables	One data-driven test per widget from the APG patterns
Role/name/state	Query by role and accessible name only (`getByRole('tab', { name: 'Billing', selected: true })`)
a11y	`jest-axe` on every story state (open, disabled, error). axe finds a subset of issues (commonly cited as a minority of WCAG failures), so it supplements, not replaces, manual keyboard and screen-reader checks (Accessibility testing)
Controlled/uncontrolled	Same test suite run twice with a harness
SSR	`renderToString` smoke test plus hydration test: no mismatch warnings; ids identical
Types	`expect-type` / `tsd`: `Button asChild` accepts `<a>`; `as="a"` requires/permits `href`; wrong props fail
Focus	Dialog restores focus to trigger; closes on Escape; trap cycles; nested dialogs
Visual regression	Storybook snapshots for each state in light/dark; review diffs (Visual regression)
Consumer install	`npm pack`, install into a fresh Vite app, import, build, run a smoke test

Example keyboard contract:

```
const cases = [
  ['ArrowRight', 'Billing'],
  ['ArrowLeft', 'Security'], // starting on the first tab, ArrowLeft wraps to the last
(verified; the original table wrongly expected 'Profile')
  ['End', 'Security'],
  ['Home', 'Profile'],
] as const

it.each(cases)('Tabs: %s moves focus and selection', async (key, expected) => {
  const user = userEvent.setup()
  render(<DemoTabs defaultValue="Profile" />)
  await user.tab()                                  // enters the tablist at the selected tab
  expect(screen.getByRole('tab', { name: 'Profile' })).toHaveFocus()
  await user.keyboard(`{${key}}`)
  expect(screen.getByRole('tab', { name: expected, selected: true })).toHaveFocus()
})
```

Dialog focus test:

```tsx
it('restores focus to the trigger on close', async () => {
  const user = userEvent.setup()
  render(<DemoDialog />)
  const trigger = screen.getByRole('button', { name: /open/i })
  await user.click(trigger)
  expect(await screen.findByRole('dialog', { name: /settings/i })).toBeInTheDocument()
  await user.keyboard('{Escape}')
  expect(screen.queryByRole('dialog')).not.toBeInTheDocument()
  expect(trigger).toHaveFocus()
})
```

jsdom limits: no layout, no real focus-visible, partial `inert` / `<dialog>` support. Cover layout-dependent behavior (positioning, scroll lock visuals) with Playwright against Storybook.

---

9. Performance expectations

Concern	Expectation
Bundle	Per-component size budgets (N2) enforced by `size-limit` in CI; importing `Button` must not include `Dialog` code
Tree-shaking	`sideEffects` set; CSS marked as side-effectful; verified in a consumer bundle
Re-renders	Typing in `TextField`, hovering a `Tooltip`, or moving the `Menu` highlight does not re-render unrelated component subtrees (profile with DevTools)
Context	Context values memoized; frequently changing values (hover index) kept out of the context that all items subscribe to
Listeners	Global listeners (Escape, outside press) attached only while open and removed on cleanup; no leaks after 1,000 open/close cycles (Memory leaks)
Layout	Positioning reads layout once per open and on scroll/resize (throttled), not on every render
Large lists	`Select` / `Menu` with 5,000 options remains responsive, or documents a virtualization strategy (Windowing and virtualization)
Compiler compatibility	Components obey the Rules of React so the React Compiler (1.0, stable) can optimize them (React Compiler); do not require consumers to memoize

---

10. Common implementation mistakes

Mistake	Consequence	Fix
`forwardRef` in a React 19 library	Needless wrapper, awkward typing	`ref` as a prop; keep `forwardRef` only if supporting React 18 (then ship dual strategy and say so)
Overriding consumer handlers (`{...rest}` placed before ours)	Consumer `onClick` silently ignored	`composeEventHandlers`; spread order matters

Mistake	Consequence	Fix
Spreading `{...rest}` before computed attributes	Consumers override `aria-*` and break semantics	Decide per attribute whether consumers may override
Prop explosion (`headerClassName` , `footerStyle` , ...)	Unbounded API	Compound parts and slots
Boolean flags that conflict (`primary` , `secondary` , `danger` all booleans)	Impossible states	A single `variant` union
Controlled/uncontrolled via `value !== undefined` while consumer passes `value={undefined}` initially	Flip warnings; lost state	Document; warn
Calling `onChange` inside a state updater	Double calls in StrictMode, side effects in render path	Compute then call
`div` with `onClick` as button	No keyboard, no role	Native elements first
Removing focus outlines	Keyboard users lost	`:focus-visible` styling
`disabled` on loading buttons	Focus lost, announcement lost	`aria-disabled` , `aria-busy`
Focus trap that never restores focus	Disorienting on close	Restore to trigger
Ids generated with `Math.random()` or a module counter	SSR hydration mismatch	`useId`
Accessing `document` during render	SSR crash	Effects, `typeof document` guards, deferred portals
Context value recreated each render	All consumers re-render each time	`useMemo` the value (or rely on compiler)
Using array index as key for dynamic Menu items	Wrong typeahead/focus after reorder	Stable keys (Lists, keys, identity)
`aria-label` duplicating visible text differently	WCAG 2.5.3 Label in Name violation	Accessible name contains visible label
Shipping CSS-in-JS runtime or global resets	Consumer style conflicts and bytes	Scoped class names + CSS vars; opt-in reset
Not externalizing `react`	Two Reacts, invalid hook call	Peer dependency + `external`
Exporting internals "just in case"	Cannot change without a major	`exports` map, explicit barrels
Tests asserting class names and DOM shape	Break on refactor	Roles, names, behavior
Treating axe-clean as accessible	Missed keyboard/SR issues	Manual keyboard pass and screen-reader smoke test per component

11. Review checklist

API design

- [ ] Three most common usage scenarios per component fit on one screen of code.
- [ ] Controlled and uncontrolled both work, with identical prop naming.
- [ ] Consumer `className`, `style`, handlers, `ref`, and data/aria attributes pass through safely.
- [ ] No prop exists that could be expressed by composition.
- [ ] Misuse produces actionable dev warnings; production stays quiet.

Accessibility

- [ ] Each component maps to an APG pattern and documents deviations.
- [ ] Full keyboard operation; visible focus; no traps except the modal trap.
- [ ] Names, roles, states exposed; ids wired with `useId`.
- [ ] Reduced motion, high zoom (reflow), and target size checked.
- [ ] Manual screen-reader check recorded for Dialog, Tabs, Menu, Toast.

Architecture

- [ ] Behavior in primitives; styling in the styled tier; the tiers do not leak.
- [ ] Internal core not exported.
- [ ] Dependency direction: styled to primitives to hooks to core.

Packaging

- [ ] `exports`, `types`, `sideEffects`, and `peerDependencies` correct.
- [ ] Verified by installing the tarball in a fresh app (ESM, types, CSS).
- [ ] Size budgets enforced.

Quality

- [ ] Contract tests per widget, SSR tests, type tests, visual baselines.
- [ ] Changesets entry per change; deprecation path demonstrated.
- [ ] Docs list props, defaults, a11y notes, and controlled examples.

---

12. Reference solution guidance

1. **Write usage before implementation (M0).** If the usage code is awkward, no implementation will rescue the API. Revise the sketches twice.
2. **Build `Tabs` completely before `Dialog`.** Tabs forces the compound-context-controllable-roving-focus stack in a small surface. Everything later reuses it.
3. **Extract hooks only at the second use.** `useRovingFocus` appears from Tabs and Menu; resist extracting it from Tabs alone (rule of three guides the abstraction level; see DRY, KISS, YAGNI).
4. **Encode complex widgets as reducers or machines.** Menu/Select/Combobox have enough states (closed, open, highlighted index, typeahead buffer, selected) that a reducer with explicit actions is easier to test than scattered `useState`s (Reducer pattern, State machine pattern):

```
type MenuState = { open: boolean; active: number | null; query: string }
type MenuAction =
  | { type: 'open'; focus: 'first' | 'last' }
  | { type: 'close' }
  | { type: 'move'; delta: 1 | -1; count: number }
  | { type: 'home' } | { type: 'end'; count: number }
  | { type: 'type'; char: string; match: (q: string) => number | null }

function reducer(s: MenuState, a: MenuAction): MenuState { /* pure; unit-tested without DOM */
}
```

5. **Run the same suite against both usage modes** (controlled harness and uncontrolled harness) via a shared test factory.
6. **Dogfood in two consumers** with different styling (styled tier in one, headless primitives in the other). If the second consumer needs a fork, the primitive API is wrong.
7. **Document decisions in ADRs** (Architecture decision records): at least polymorphism, Dialog implementation, positioning, styling approach.
8. **Release process:** each PR includes a changeset; CI runs build, type tests, unit/a11y tests, size-limit, and the consumer install test. A breaking change requires a migration note.
9. **Demonstrate a deprecation:** rename `Tooltip`'s `delay` prop to `openDelay`; keep `delay` for one minor with a dev warning and a changeset note:

```
/** @deprecated Use `openDelay`. Will be removed in the next major. */
delay?: number
// in the component:
warn(props.delay === undefined, '`delay` is deprecated; use `openDelay`.')
const openDelay = props.openDelay ?? props.delay ?? 700
```

---

13. Optional extensions

- **Combobox with async options** and virtual focus; compare with Select (Debounce and throttle for filtering).
- **Date picker** (grid keyboard pattern, locale-aware) (Internationalization).
- **Form integration:** `TextField` / `Select` participating in native form submission and React 19 form Actions via hidden inputs and `ElementInternals`-free fallbacks (Forms, actions and mutations).
- **RTL and i18n support** across all keyboard handling.
- **Monorepo packaging:** split `@acme/primitives`, `@acme/ui`, `@acme/tokens` with a workspace and a shared config (Monorepos and shared packages).
- **Design tokens pipeline** (JSON tokens compiled to CSS variables + TypeScript types) (Design system architecture).
- **Codemod** for the demonstrated deprecation using `jscodeshift` or a TypeScript-based transformer.
- **Compiler + library study:** run the library through the React Compiler's lint/checks (the compiler rules ship in `eslint-plugin-react-hooks`) and fix violations of the Rules of React.

- **Governance simulation:** write a contribution process, a component proposal template, and an accessibility review gate (Design system governance, Code review and standards).